Node.js

C’est Quoi Node.js

1. Node.je est un environnement côté serveur, open source
2. Node.js est multiplatform
3. Node.js utilise du javascript côté serveur

Pourquoi Node.js

- Node.js utilise une programmation ***Asynchrone***
- Node.js est mono-Thread non-blocking.
- Un programme Node.js exécute les instructions sans arret et garantie un fonctionnement non bloquant.
- Node.js utilise le moteur d'exécution ultrarapide V8 de Google Chrome.
- Ce moteur est caractérisé la compilation JIT (Just In Time). Il transforme le code JavaScript très rapidement en code machine.

Pourquoi Node.js

Pour lire un fichier dans le serveur

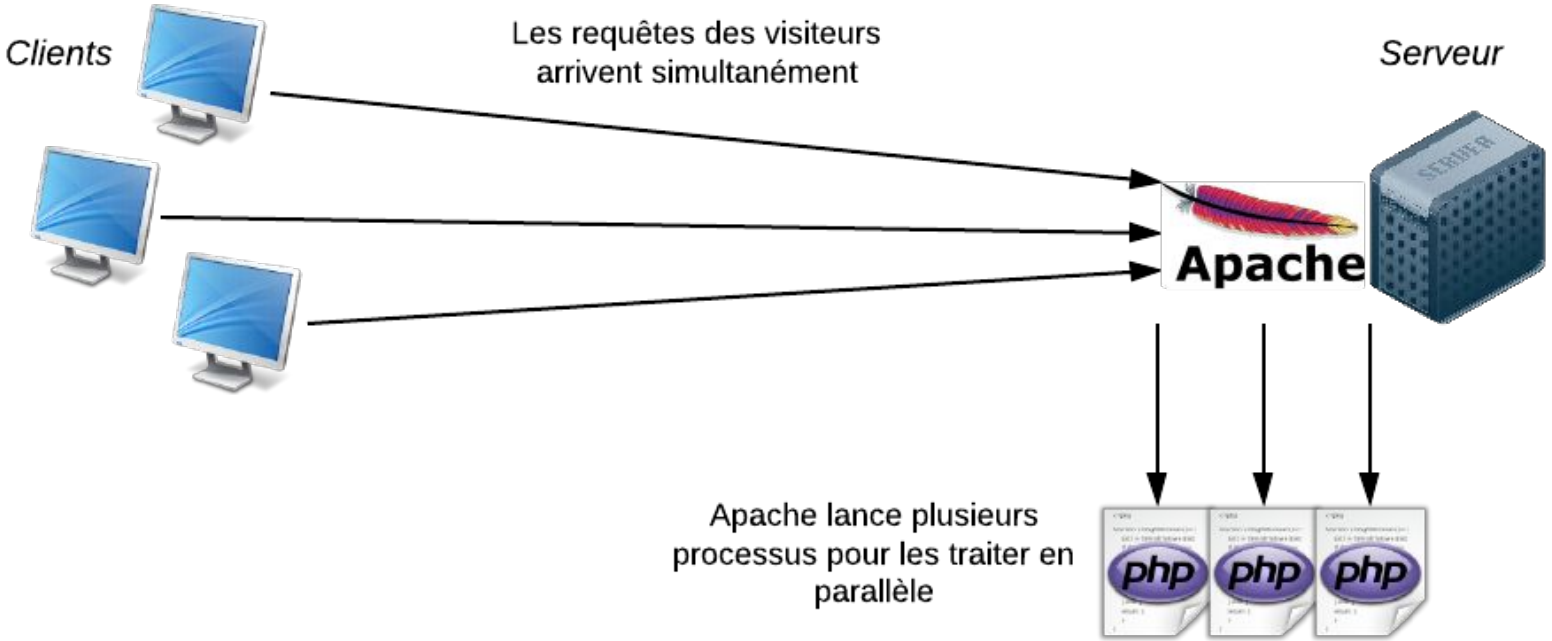
PHP

1. Envoyer la requête au serveur
2. Demander au système de fichier d'ouvrire le fichier
3. **Attendre l'ouverture du fichier**
4. Envoyer le contenue au client
5. Passer à la requête suivante

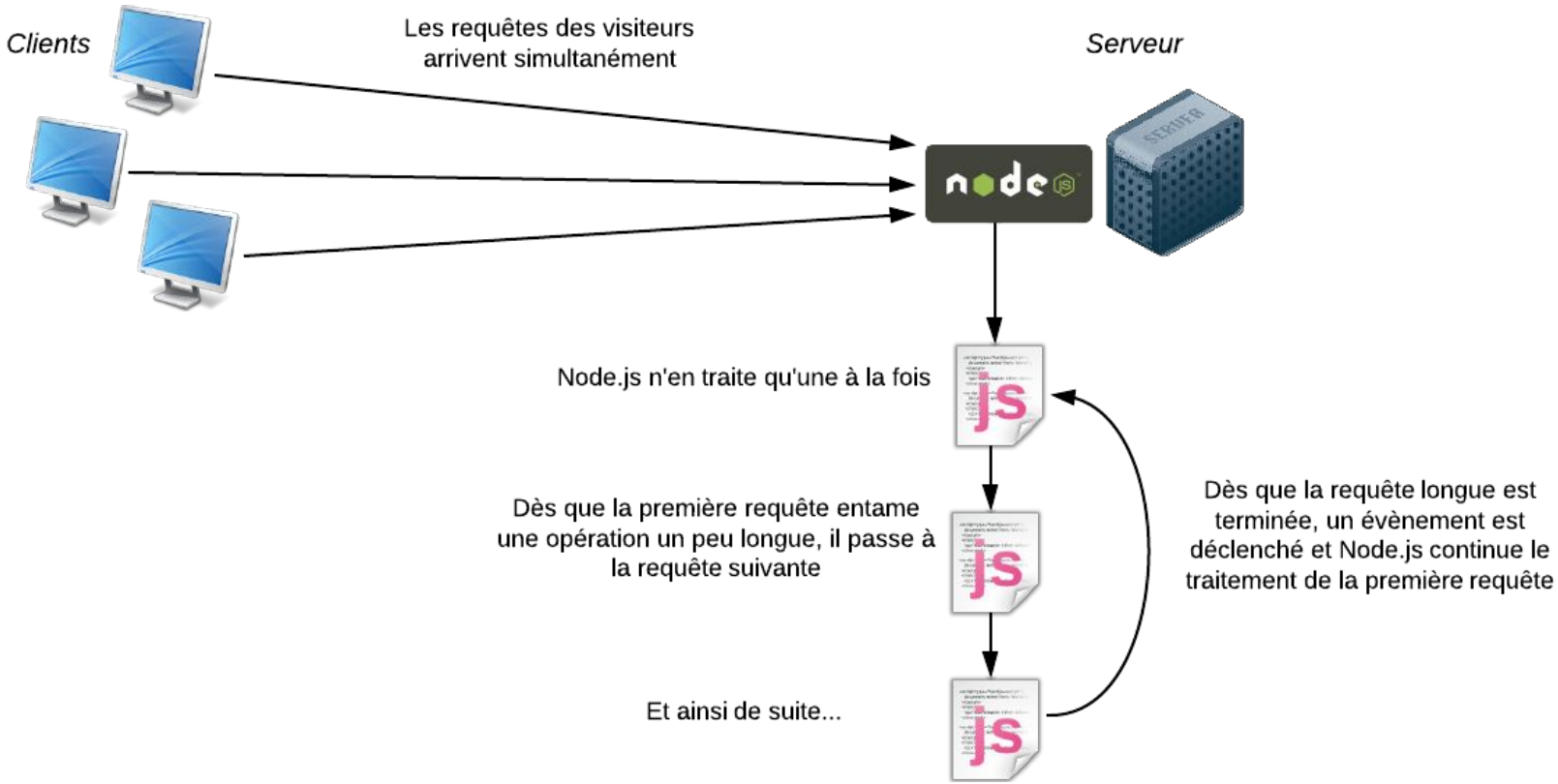
Node.js

1. Envoyer la requête au serveur
2. Demander au système de fichier d'ouvrire le fichier
3. Passer à la requête suivante
4. Quand le contenu du fichier est prêt Envoyer le contenue au client

Pourquoi Node.js



Pourquoi Node.js



Que peut faire Node.js

- Générer des page de contenue dynamique
- Créer, lire, modifier, supprimer des fichiers dans le serveur
- Créer, lire, modifier, supprimer des données dans une base de données
- Accéder au resource matériel du serveur (wifi, bluetooth, webcam ...)
- ...

Premier script Node.js

fileReader.js

```
1  var fs = require('fs'),
2    path = './fichier.txt';
3
4  console.log('before');
5
6  fs.readFile(path, function(err, file) {
7    console.log('during');
8    console.log('' + file);
9  });
10
11 console.log('after');
```

- ***Resultat !!!***

```
gnu@gnu-ThinkPad-T420s:~/EST/ISIL/WEB/Nodejs/01$ node fileReader.js
before
after
during
ceci est un texte de './fichier.txt'
```

Premier script Node.js

nodejsApp.js

```
1  var http = require('http');
2
3  var server=http.createServer(function (req, res) {
4      res.writeHead(200, {'Content-Type': 'text/html'});
5      res.write('<h1>Bonjour ISIL</h1>');
6      res.write('<h3>Powered by node.js</h3>');
7      res.end('...Fin');
8      });
9
10 server.listen(3000);
```

- Pour lancer l'application (le serveur HTTP)
  - $ ***node nodejsApp.js***
- Pour visualiser le résultat dans le navigateur
  - ***http://localhost:3000/***

Premier script Node.js

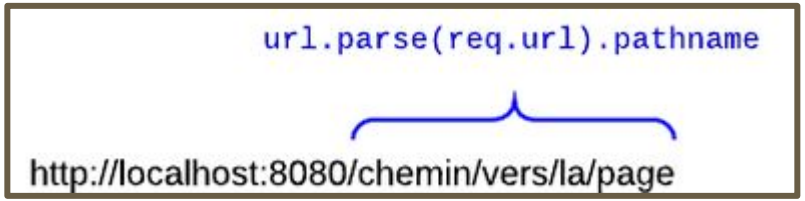
content-type peut être:

- Du texte brut : **text/plain**
- Du HTML : **text/html**
- Du CSS : **text/css**
- Une image JPEG : **image/jpeg**
- Une vidéo MPEG4 : **video/mp4**
- Un fichier ZIP : **application/zip**
- etc.

Node.js: Gestion des requêtes

Pour récupérer la page demandée par le visiteur:

1. on utilise le module "***url***" pour "parser" la requête
2. Extraire La page en question (***pathname***)
3. Envoyer la réponse adequate



Node.js: Gestion des requêtes

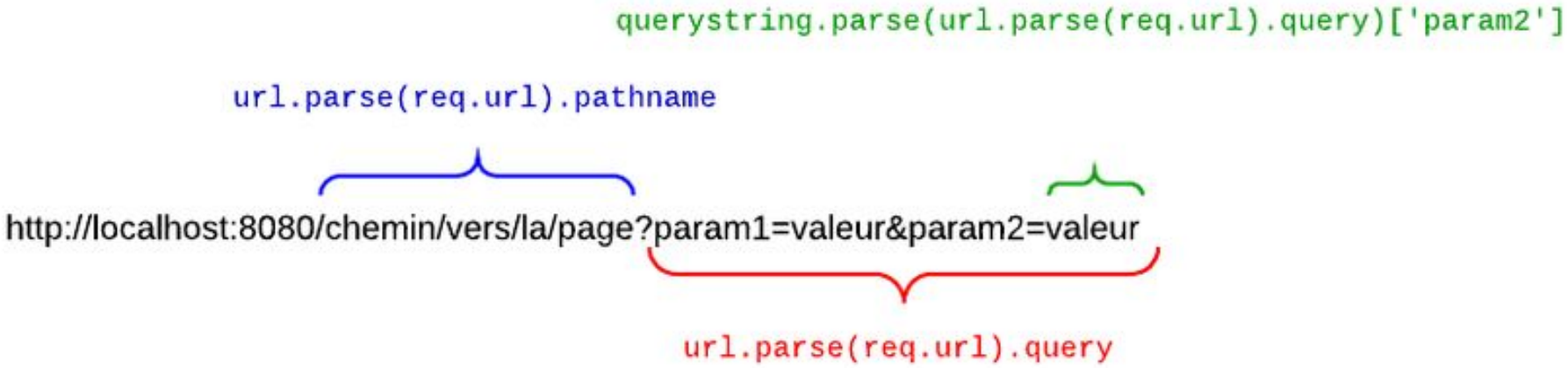
```
1  var http = require('http');
2  var url = require('url');
3
4  var server = http.createServer(function(req, res) {
5      var page = url.parse(req.url).pathname;
6      console.log(page);
7      res.writeHead(200, {"Content-Type": "text/plain"});
8      if (page == '/') {
9          res.write('Accueil');
10     }
11     else if (page == '/contact') {
12         res.write('Nous contscter');
13     }
14     else if (page == '/cours/web/nodejs') {
15         res.write('Node.js is greate');
16     }
17     res.end();
18 });
19
20 server.listen(3000);
```

Node.js: Gestion des requête

Pour récupérer les paramètres de la requête:

1. Utiliser le module "***url***" pour "parser" la requête
2. Extraire Les paramètres en question (***query***) => chaine de caracteres
3. Utiliser le module “***querystring***” pour “parser” cette chaîne en {key: value}
4. Envoyer la réponse adequate

Node.js: Gestion des requête



Node.js: Gestion des requête

```
1  var http = require('http');
2  var url = require('url');
3
4  var querystring = require('querystring');
5
6  var server = http.createServer(function(req, res) {
7      var params = querystring.parse(url.parse(req.url).query);
8      res.writeHead(200, {"Content-Type": "text/plain"});
9      if ('prenom' in params && 'nom' in params) {
10         res.write('Vous vous appelez ' + params['prenom'] + ' ' + params['nom']);
11     }
12     else {
13         res.write('Il manque des parametres!!');
14     }
15     res.end();
16
17 });
18
19 server.listen(3000);
```

1. Lancer le serveur: ***$ node params.js***
2. Envoyer la requete : ***http://localhost:3000/?nom=yahyaoui&prenom=anas***
3. Puis la requete: ***http://localhost:3000***

Node.js: les evenements

- On Node.js, un grand nombre d'objets émettent des évènements.
- Ces objets héritent tous d'un objet EventEmitter fourni par Node.
- Par exemple L’objet **server** de **HTTP** possède les événements:
  - ***Request***
  - ***Connection***
  - ***Close***
  - ***Connect***
  - ***Upgrade***
  - ***clienError***

Node.js: les evenements

- Pour écouter à un événement sur un objet on appel la méthode on() de cet objet:

*obj.**on**(‘**evenement**’, **callbackFunction**)*

Node.js: les evenements

- Pour écouter à un événement sur un objet on appel la méthode on() de cet objet:

*obj.**on**('**evenement**', **callbackFunction**)*

```
var server = http.createServer();
server.on('request', function(req, res) { });
```

```
var server = http.createServer(function(req, res) { });
```

Node.js: les evenements

```
1  var http = require('http');
2
3  var server = http.createServer(function(req, res) {
4    res.writeHead(200);
5    res.end('Salut ISIL !');
6  });
7
8  server.on('close', function() { // On écoute l'évènement close
9      console.log('Au revoir!!');
10 });
11
12 server.listen(3000); // Démarre le serveur
13 server.close(); // Arrête le serveur. Déclenche l'évènement close
14
```

Node.js: les evenements

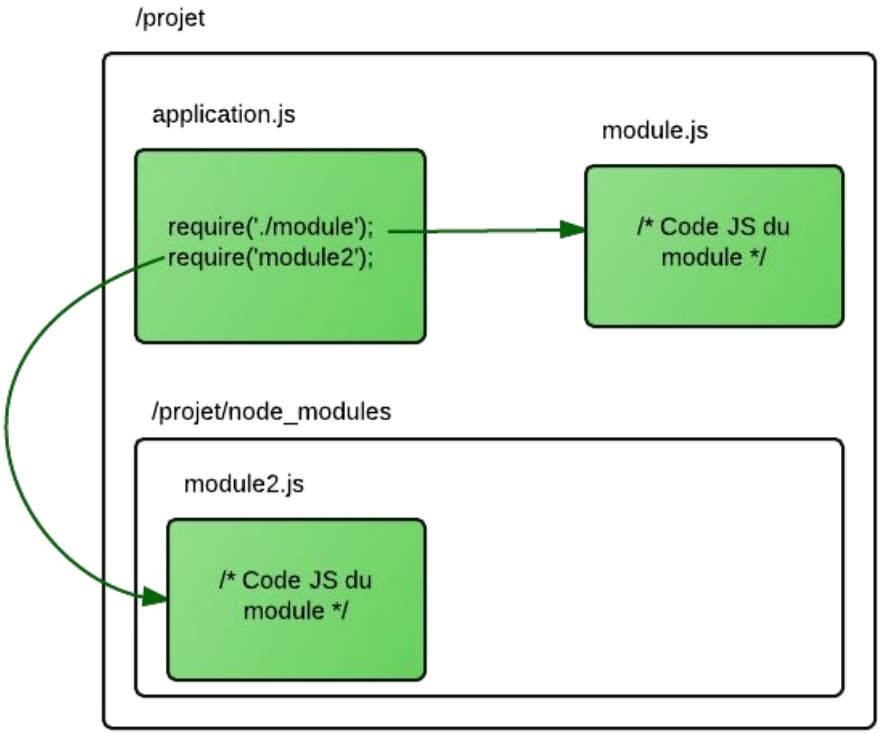
- Pour créer un objet susceptible d'émettre un événement:
- Inclure le module '***events***'
- Créez un objet basé sur EventEmitter
- Quelque part dans l'app, Pour écouter l'événement sur cet objet
  - *obj.**on('nomEvent', function(param1,param2, ...){ ... }**)*
- Pour déclencher l'événement on appeler la méthode ***emit('nomEvent', param1, param2, ...)***

Node.js: les evenements

```
1  var EventEmitter = require('events').EventEmitter;
2
3  var monObjet = new EventEmitter();
4
5  monObj.on('monEvent', function(param){// on ecoute monEvent
6      console.log(param);
7
8  });
9
10 monObje.emit('monEvent', 'Un texte de monEvent');// Declancher monEvent
```

Les modules

- Le noyau de Node.js est très petit.
- Ne peut pas faire grand chose
- Mais tres riche on ***modules***



Les modules

- Chercher un module
  - $***npm search nomModule***
- Installer un module
  - $***npm install nomModule*** //installation locale
  - $***npm install nomModule -g*** //installation globale
  -
- Metre a jour un module
  - $***npm update nomModule***
- Publier un module
  - $npm adduser //Creer un compte su npm
  - npm publish // publier le module

```
1  var monmodule = require('./monmodule');
2
3  monmodule.direBonjour();
4  monmodule.direByeBye();
```

```
1  var direBonjour = function() {
2      console.log('Bonjour !');
3  }
4
5  var direByeBye = function() {
6      console.log('Bye bye !');
7  }
8
9  exports.direBonjour = direBonjour;
10 exports.direByeBye = direByeBye;
```

application.js

```
var monmodule = require('./monmodule');

monmodule.mafonction();
```

module.js

```
var mafonction = function() { ... };

exports.mafonction = mafonction;
```

Express.js

Le framework Express.js

- Express.js est un micro-framework pour Node.js.
- Il met à disposition des outils de base pour aller plus vite dans la création d'applications Node.js.
- Pas assez complet comme les framework php (laravel, symphony ..) python(django).
- Pour installer express.js
  - ***npm install express***

Express.js: firstApp

```
var http = require('http');
var url = require('url');

var server = http.createServer(function(req, res) {
    var page = url.parse(req.url).pathname;
    console.log(page);
    res.writeHead(200, {"Content-Type": "text/plain"});
    if (page == '/') {
        res.write('Accueil');
    }
    res.end();
});

server.listen(3000);
```

```
var express = require('express');

var app = express();

app.get('/', function(req, res) {
    res.setHeader('Content-Type', 'text/plain');
    res.send('Accueil');
});

app.listen(3000);
```

Les routes

```
1  var express = require('express');
2  var app = express();
3
4  app.get('/', function(req, res) {
5      res.setHeader('Content-Type', 'text/plain');
6      res.write('Accueil');
7  });
8
9  app.get('/contact', function(req, res) {
10     res.setHeader('Content-Type', 'text/plain');
11     res.write('Nous contscter');
12 });
13
14 app.get('/cours/web/nodejs', function(req, res) {
15     res.setHeader('Content-Type', 'text/plain');
16     res.write('Node.js is greate');
17 });
18
19 app.use(function(req, res, next){
20     res.setHeader('Content-Type', 'text/plain');
21     res.status(404).send('Page introuvable !');
22 });
23
24 app.listen(3000);
```

Les routes dynamiques

Express permet de gérer des routes dont certaines portions peuvent varier.

En inserant une variable ***:mavariable*** dans l'URL de la route

Ce qui crée un paramètre accessible avec ***req.params.mavariable***.

```
26  app.get('/capteurs/:id/:temp', function(req, res) {
27      res.setHeader('Content-Type', 'text/plain');
28      res.end('La temperature mesurerpar le capteur n°' +
29              req.params.id + ' est: ' + req.params.);
30  });
```

les middlewares

- Ce sont des petits morceaux d'application qui rendent chacun un service spécifique.
- Les middlewares de base dans Express ne sont pas nombreux:
  - ***compression*** : permet la compression gzip de la page
  - ***cookie-parser*** : permet de manipuler les cookies
  - ***cookie-session*** : permet de gérer des informations de session (durant la visite d'un visiteur)
  - ***serve-static*** : permet de renvoyer des fichiers statiques contenus dans un dossier (images, fichiers à télécharger...)
  - ***serve-favicon*** : permet de renvoyer la favicon du site
  - etc.

les middlewares

- Ces middlewares sont interconnectés et peuvent communiquer entre eux en se renvoyant jusqu'à 4 paramètres
  - ***err***: les erreurs
  - ***req***: la requête du visiteur
  - ***res***: la réponse à renvoyer (la page HTML et les informations d'en-tête)
  - ***next***: un callback vers la prochaine fonction à appeler

les middlewares

```
1  var express = require('express');
2  var favicon = require('serve-favicon'); // Charge le middleware de favicon
3
4
5  var app = express();
6  // Indiquer que le dossier /public contient des fichiers
7  //statiques (middleware chargé de base)
8  app.use(express.static(__dirname + '/public'));
9
10 // Activer la favicon indiquée
11 app.use(favicon(__dirname + '/public/favicon.ico'));
12
13 // Répondre a la requete
14 app.use(function(req, res){
15     res.send('Hello');
16 });
17
18 app.listen(3000);
```

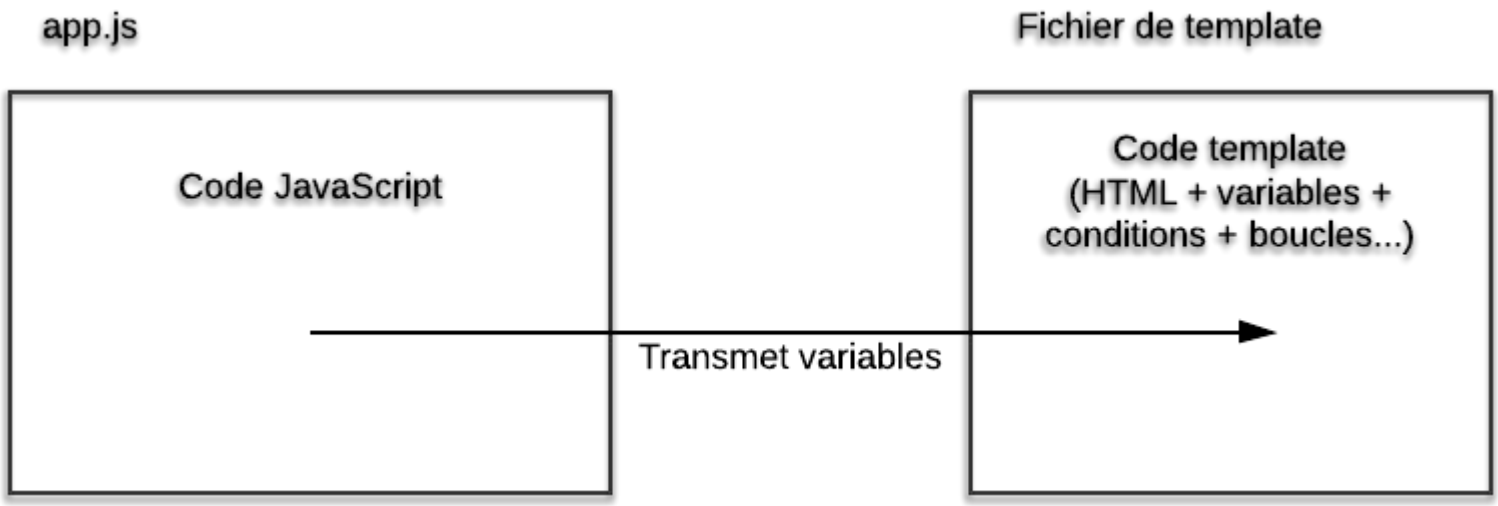
Les templates

- Envoyer un contenu Html en utilisant cette méthode n’est pas une tâche agréable
- Les templates permettent de produire du HTML et d'insérer au milieu du contenu variable(javascript).
- Il existe beaucoup de langages de templates: Twig, Smarty, Haml, JSP, Jade, EJS...

```
1  res.write('<!DOCTYPE html>'+
2  '<html>'+
3  '    <head>'+
4  '        <meta charset="utf-8" />'+
5  '        <title>Ma page Node.js !</title>'+
6  '    </head>'+
7  '    <body>'+
8  '       <p>Voici un paragraphe <strong>HTML</strong> !</p>'+
9  '    </body>'+
10 '</html>');
```

Les templates

depuis un fichier JavaScript, on appelle un template en lui transmettant les variables dont il a besoin pour construire la page.



EJS

EJS : Embedded JavaScript